Graphical Models 141 (2025) 101301

Contents lists available at ScienceDirect

Graphical Models

journal homepage: www.elsevier.com/locate/gmod

Edge-aware denoising framework for real-time mobile ray tracing

Haosen Fu *, Mingcong Ma, Junqiu Zhu, Lu Wang, Yanning Xu

Shandong University, China

ARTICLE INFO

Keywords:
Edge-aware kernel size adaptation
Weight prediction
Edge density reuse

ABSTRACT

With the proliferation of mobile hardware-accelerated ray tracing, visual quality at low sampling rates (1spp) significantly deteriorates due to high-frequency noise and temporal artifacts introduced by Monte Carlo path tracing. Traditional spatiotemporal denoising methods, such as Spatiotemporal Variance-Guided Filtering (SVGF), effectively suppress noise by fusing multi-frame information and using geometry buffer (G-buffer) guided filters. However, their reliance on per-frame variance computation and global filtering imposes prohibitive overhead for mobile devices. This paper proposes an edge-aware, data-driven real-time denoising architecture within the SVGF framework, tailored explicitly for mobile computational constraints. Our method introduces two key innovations that eliminate variance estimation overhead: (1) an adaptive filtering kernel sizing mechanism, which dynamically adjusts filtering scope based on local complexity analysis of the G-buffer; and (2) a data-driven weight table construction strategy, converting traditional computational processes into efficient real-time lookup operations. These innovations significantly enhance processing efficiency while preserving edge accuracy. Experimental results on the Qualcomm Snapdragon 768G platform demonstrate that our method achieves 55 FPS with 1spp input. This **frame rate is 67.42% higher** than mobile-optimized SVGF, provides **better visual quality**, and **reduces power consumption by 16.80%**. Our solution offers a practical and efficient denoising framework suitable for real-time ray tracing in mobile gaming and AR/VR applications.

1. Introduction

Recent advances in mobile GPU architectures have begun to make real-time ray tracing feasible on mobile platforms. Notably, SoCs such as the Qualcomm Snapdragon 8 Gen3 and MediaTek Dimensity 9300 now include hardware support for ray traversal and intersection, enabling physically based rendering effects on consumer-grade devices. However, due to stringent constraints on power and thermal dissipation, the sampling rate for mobile ray tracing remains severely limited, often below 1 spp. This leads to pronounced high-frequency Monte Carlo noise and severe temporal artifacts such as ghosting and flickering in rendered frames.

Among existing real-time denoising techniques, SVGF [1] has proven effective on desktop platforms due to its use of temporal accumulation and feature-aware spatial filtering. However, SVGF relies on large fixed-size filter kernels and per-pixel floating-point variance estimation, resulting in significant computational overhead when deployed on mobile GPUs. To overcome these limitations, we propose a mobile-optimized denoising framework that integrates kernel adaptation and neural approximation to reduce cost while preserving image fidelity.

Our method introduces an edge-aware adaptive filtering scheme that modulates the spatial filter kernel size dynamically, based on a compact edge density representation derived from G-buffer features. By clustering multi-channel edge responses, we generate a density map that contracts the filter in regions with high geometric complexity and expands it in flatter areas. This localized kernel adjustment reduces unnecessary computations in uniform regions while preserving structural details at edges.

To further reduce runtime cost and eliminate dependence on expensive floating-point arithmetic, we replace SVGF's strict weight computation with a data-driven strategy. Specifically, we train a lightweight neural network to predict per-pixel filter weights from a reduced feature set, effectively serving as a learned surrogate for SVGF's spatial weighting function. The predicted weights are stored in a compact lookup table, enabling efficient inference via table indexing during rendering.

We additionally enhance temporal stability by incorporating an improved spatiotemporal reuse strategy. Motion vectors are used to reproject edge density maps from previous frames, and hole-filling techniques ensure consistency across time steps. This results in temporally coherent filtering without the overhead of full-frame recalculation.

The main contributions of this work are as follows:

* Corresponding author.
E-mail address: fhs18391010985@163.com (H. Fu).

https://doi.org/10.1016/j.gmod.2025.101301
Received 26 June 2025; Received in revised form 25 July 2025; Accepted 9 August 2025
Available online 27 August 2025
1524-0703/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

- Edge-aware kernel adaptation: a dynamic filter sizing strategy that modulates spatial kernels according to local edge complexity, improving efficiency over fixed-kernel SVGF.
- Neural weight approximation: a compact, learned model that replaces floating-point filtering weights with precomputed lookups derived from feature maps.
- Temporal reuse mechanism: a motion-compensated edge density fusion method that enhances inter-frame coherence with minimal computational cost.

Our method achieves real-time denoising on mobile devices at interactive frame rates, while maintaining visual quality comparable to SVGF on desktop-class hardware. The proposed system provides a practical solution for integrating ray-traced effects into mobile applications without exceeding their performance budgets.

2. Related work

Monte Carlo ray tracing approximates given scenes through the accumulation of stochastic samples. Extensive research has focused on reconstructing accurate results from noisy images rendered with low sampling rates, including techniques such as guided blur kernels, machine learning-driven filters, and spatiotemporal accumulation schemes [2]. However, most of these methods are applied in offline rendering, requiring seconds or even minutes to complete image reconstruction.

The core objective of classical ray tracing denoising lies in enhancing the visual quality of low-sampled images through spatiotemporal information fusion, primarily revolving around filter kernel design and temporal accumulation strategies. Spatial-domain methods often employ filtering techniques such as Gaussian [3], Bilateral [4], A-Trous [5], Guided [6], and Median [7] to blur Monte Carlo-rendered images. In particular, guided filter applications driven by feature buffers (such as G-buffer in deferred rendering, first-bounce data, reprojected path length, or view position) are the most widely used. Concurrently, advancements in temporal accumulation have spurred new denoising algorithms. For instance, Blockwise Multilevel Filtering and Reprojection (BMFR) [8] introduced a blockwise motion compensation strategy, dividing images into sub-blocks for independent motion estimation to alleviate blocking artifacts. SVGF achieved efficient noise suppression on desktop platforms by leveraging variance estimation and blending geometric features (e.g., normals, depth) with historical frames. A-SVGF [9] further improved SVGF's temporal filtering by dynamically computing per-pixel temporal blending coefficients using estimated temporal gradients instead of fixed parameters, enhancing temporal stability. Despite their effectiveness, these algorithms suffer from high computational intensity, where exponential operations and conditional branches create severe instruction throughput bottlenecks on mobile platforms. Additionally, their reliance on rigid analytical formulas necessitates scene-specific parameter tuning, making them impractical for real-time mobile applications.

With the rise of machine learning, neural network-based denoising methods have emerged, focusing on end-to-end learning of mappings from low-sampled inputs to high-quality outputs. KPCN pioneered pixel-wise custom filter kernel prediction through implicit feature weight learning via convolutional layers, excelling in handling complex materials (e.g., fabrics, hair) but requiring intensive computation for full-resolution kernel prediction. Kalantari et al. employed multilayer perceptrons(MLP) [10] to optimize parameters for joint bilateral and non-local means denoisers applied to Monte Carlo noise. Chaitanya et al. integrated recurrent connections (LSTM/GRU) [11] with historical frame information to suppress inter-frame flickering and improve dynamic scene stability. Vogels et al. fused multi-scale features via skip connections and penalized over-smoothing using asymmetric loss functions [12]. Xu et al. proposed GAN-based [13] Monte Carlo denoising, leveraging U-Net generators and adversarial training to synthesize realistic details. However, existing data-driven methods face inherent trade-offs between model complexity and real-time performance on mobile devices, with limited optimization for mobile GPU architectures. Although subsequent efforts like Real-Time Denoising Network (RTDNet) [14] adopted U-Net architectures with depthwise separable convolutions to minimize parameters, they remain insufficient for mobile real-time requirements.

Current mobile denoising solutions face efficiency and generalization challenges. For example, Qualcomm's Shadow Denoiser employs a 4×4 accelerated dedicated hardware unit for real-time filtering via precomputed light field features but supports only specific light types [15]. OPPO's Hybrid Rendering Engine combines rasterization pipelines with ray-traced denoising [16], reducing power consumption through dynamic workload allocation but incurring a 15% loss in material reflection accuracy. These examples highlight the ongoing struggle to balance computational efficiency and visual fidelity in mobile denoising.

3. Overview of the spatial filtering pipeline

We propose a real-time denoising pipeline tailored for 1spp ray-traced images on mobile hardware. The system leverages G-buffers containing surface normals, depth, and albedo, as well as motion vectors and temporal edge density information to perform spatially adaptive denoising while preserving fine details. Fig. 1 illustrates the overall pipeline architecture, which integrates edge-guided kernel adaptation and a lightweight, data-driven weight prediction module for efficient denoising under tight compute and memory budgets.

Edge-Aware Kernel Size Adaptation. The pipeline begins by computing a per-pixel edge map from the current frame's G-buffers. Specifically, edge likelihoods are estimated by aggregating local contrast cues across the normal, depth, and albedo channels. In parallel, edge density information from the previous frame is reprojected using per-pixel motion vectors to produce a temporally aligned warped edge density map and a corresponding hole mask indicating invalid regions.

We then calculate the edge density of the current frame by measuring the proportion of edge pixels within a local $r \times r$ window around each pixel in non-hole regions. This per-pixel edge density reflects the structural complexity of the local area. The generated edge density map is then fused with historical edge density maps to fill holes, yielding a temporally stable edge density field.

Using this fused map, we perform a multi-scale kernel grouping, in which pixels are dynamically assigned to filter kernels of varying sizes. The assignment follows an inverse relationship: regions with high edge density receive small, detail-preserving kernels, while smoother regions are assigned larger kernels to maximize denoising efficiency. This grouping strategy enables spatial adaptation to both geometry and texture complexity.

Depth-Guided Clustering and Weight Prediction. To further enhance kernel specificity, each kernel group undergoes a secondary clustering step based on local depth gradients. This clustering accounts for geometric continuity and surface orientation, ensuring that only pixels belonging to similar surfaces are processed together. The clustering process is initialized via K-means using gradient direction and magnitude as features, and refined at runtime by incorporating learned weight priors.

For each resulting cluster, filtering weights are predicted using a compact MLP trained offline. The network maps low-dimensional contextual features (e.g., geometric variance, intensity contrast) to filter weights, which are discretized and stored in a lookup table (LUT) for efficient inference. During denoising, the appropriate weights are retrieved via index-based access, significantly reducing the cost of evaluating filter kernels in real time.

Final Filtering and Output. The final denoised output is generated by applying the adaptive filter using the predicted kernel size and LUT-based weights for each pixel. This spatial filtering stage is fully

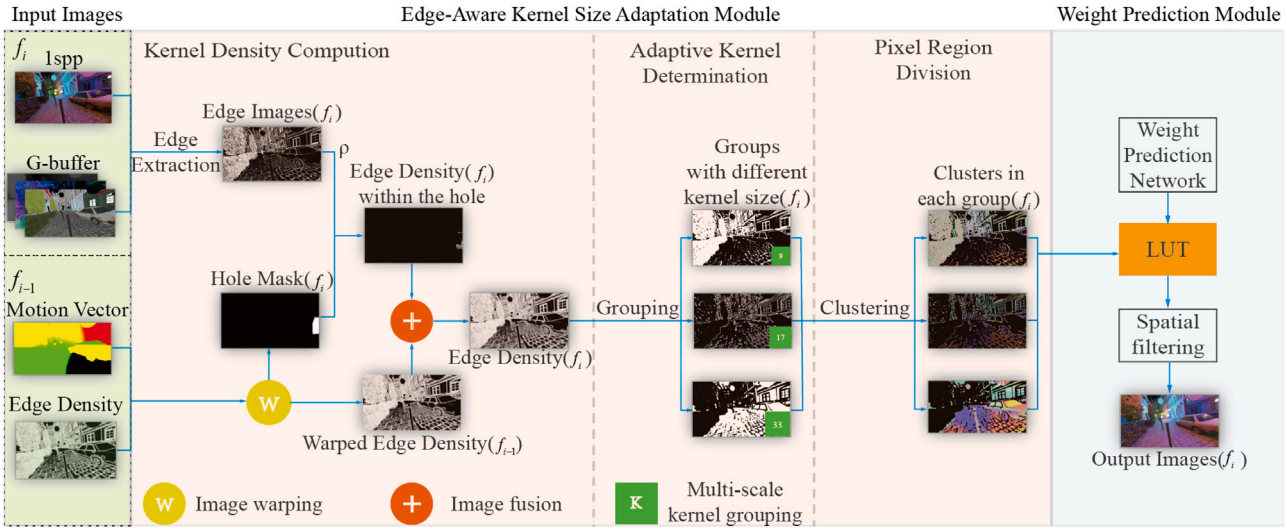


Fig. 1. Spatial filtering pipeline. Given a 1spp ray-traced input image, the system performs edge-aware kernel size adaptation and weight prediction guided by G-buffer features, motion vectors, and historical edge density maps. The final denoised output is produced by adaptive spatial filtering.

parallelized and operates in a single pass over the image. The resulting image demonstrates strong noise suppression and temporal stability while preserving structural edges and fine details, making it suitable for deployment in mobile ray tracing applications.

4. Edge-aware kernel size adaptation

This section presents an adaptive denoising framework that dynamically adjusts filter kernel sizes based on edge density. Our framework consists of three core modules: kernel density computation, adaptive kernel determination, and pixel region division. First, the framework extracts image edges through multi-channel edge detection (combining color, depth, and normal buffer features). It then computes the edge density of the current image by incorporating historical frame information, where the density value directly reflects the complexity of regional features. Based on the density distribution, the framework employs a grouping strategy to automatically select the optimal filter kernel size, mapping each pixel's edge density level to a predefined kernel scale.

To further optimize denoising quality, the framework introduces a depth-guided clustering mechanism that partitions pixels with similar geometric attributes into homogeneous regions. Each resulting cluster is then fed into a dedicated weight prediction architecture for network training (see Section 5). This edge-adaptive filtering strategy achieves an optimal balance between preserving high-frequency details and effectively suppressing noise while improving denoising efficiency.

4.1. Kernel density computation

Edge Extraction. Our framework begins with robust edge extraction from the G-buffer's Albedo, depth, and normal channels. We employ the Sobel operator [17] – a computationally efficient edge detector balancing gradient sensitivity and noise resilience – to compute directional gradients across all three channels. For each channel $c \in$ { Albedo, Depth, Normal }, horizontal (G_x^c) and vertical (G_y^c) gradients are computed using 3×3 Sobel kernels. The gradient magnitude for each channel is then calculated as:

$$M^c = \sqrt{(G_x^c)^2 + (G_y^c)^2}. \tag{1}$$

where G_x^c and G_y^c encode directional intensity variations. These magnitudes encode distinct edge semantics: Albedo gradients detect abrupt material transitions (e.g., metal-to-non-metal boundaries), depth gradients highlight geometric discontinuities (e.g., object silhouettes), and

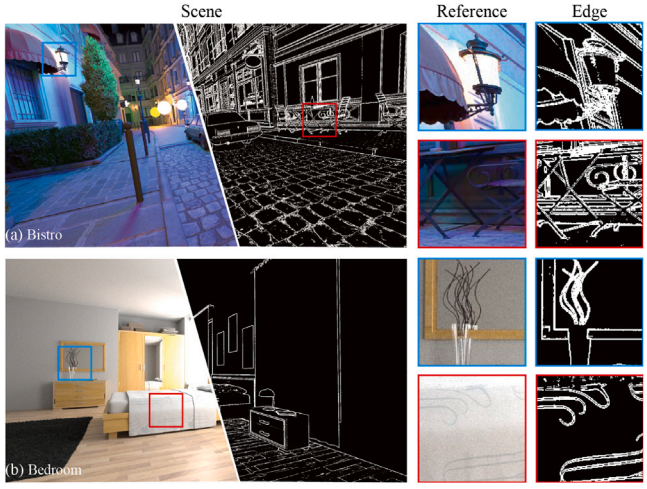


Fig. 2. Edge extraction results across two representative scenarios: (a) subtle surface patterns with low contrast gradients, and (b) fine geometric structures containing high-frequency details. Our method accurately captures both edge types.

normal gradients capture surface curvature variations (e.g., tessellation artifacts). To synthesize a robust edge prior, the magnitudes are normalized to $\tilde{M}^c \in [0, 1]$ and fused through a weighted combination:

$$M_{\text{fused}} = 0.5\tilde{M}^{\text{Albedo}} + 0.3\tilde{M}^{\text{Depth}} + 0.2\tilde{M}^{\text{Normal}}, \tag{2}$$

where the weights prioritize material edges while suppressing spurious depth noise. The fused edge image not only drives adaptive kernel sizing (see Section 4.2) but also provides gradient directionality(see Section 4.3), which informs feature clustering to enhance spatial coherence.

Following gradient computation, edges manifest as high-magnitude gradients while non-edge regions exhibit low magnitudes. To amplify edge features and suppress noise, we propose an adaptive thresholding strategy driven by global gradient statistics. By applying an adaptive threshold T to binarize the fused edge image M_{fused} (see Fig. 2), this method addresses the limitations of conventional fixed-threshold methods, which fail to adapt to varying gradient distributions under

diverse lighting and material conditions:

$$T = \mu + k\sigma, \quad \text{with} \begin{cases} \mu = \frac{1}{N}\sum_{i,j} M_{\text{fused}}(i,j) \\ \sigma = \sqrt{\frac{1}{N}\sum_{i,j}\left(M_{\text{fused}}(i,j) - \mu\right)^2} \end{cases} \tag{3}$$

where N denotes total pixels, $k = 1.2$ balances edge and noise trade-offs. The binary edge mask M_{edge} is then:

$$M_{\text{edge}}(i,j) = \begin{cases} 1 & \text{if } M_{\text{fused}}(i,j) \geq T, \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

This statistically driven method robustly segments perceptually salient edges while suppressing Monte Carlo noise, adapting to scene-specific gradient distributions, as illustrated in Fig. 2.

Temporal Edge Density Reuse. Following edge extraction, we compute the edge density to quantify local edge concentration, which serves as a key indicator for adaptive kernel sizing in subsequent denoising stages.(see Section 4.2) For each pixel (i, j), we define its edge density ρ as the proportion of edge pixels within a local neighborhood $\mathcal{N}(i, j)$:

$$\rho(i,j) = \frac{1}{|\mathcal{N}(i,j)|} \sum_{(x,y)\in\mathcal{N}(i,j)} M_{\text{edge}}(x,y), \tag{5}$$

where $\mathcal{N}(i, j)$ is an $r \times r$ window centered at (i, j), and $|\mathcal{N}(i, j)|$ denotes its area. This density metric quantifies edge sparsity, with $\rho \to 0$ indicating edge clusters and $\rho \to 1$ denoting homogeneity.

To improve the computational efficiency of edge density estimation, we leverage motion vectors from temporal reprojection to reuse edge information from previous frames in static or rigidly moving regions. For a pixel (i, j)in frame t, its corresponding position in frame $t - 1$ is given by:

$$(i', j') = (i, j) + \mathbf{V}_t(i, j), \tag{6}$$

where $\mathbf{V}_t(i, j)$ is the motion vector. If (i', j') falls within valid bounds (i.e., does not exceed the previous frame's boundaries), we directly reuse the edge density value $\rho^{t-1}(i', j')$ from the previous frame to avoid redundant computation. Otherwise, the position is marked as hole and recalculated. Finally, we generate the complete edge density image for the current frame by fusing the warped edge density image with real-time computation results. This strategy significantly maintains high fidelity while improving computational efficiency in real-time rendering scenarios requiring high temporal coherence.

4.2. Adaptive kernel determination

We propose an edge density-based grouping strategy that dynamically partitions pixels into computational groups with varying kernel sizes according to the edge density ρ. This adaptive allocation ensures areas with complex structures receive refined processing while enabling efficient large-scale filtering for homogeneous regions. The kernel size k for each group is determined as:

$$k = \begin{cases} 9 & \text{if } \rho \geq 0.5, \\ 17 & \text{if } 0.3 \leq \rho < 0.5, \\ 33 & \text{otherwise.} \end{cases} \tag{7}$$

Thresholds (0.5, 0.3) were optimized through grid search on our dataset to maximize the harmonic mean of denoising quality (PSNR) and computational efficiency (FPS). Pixels are accordingly divided into three distinct processing groups (with 9×9, 17×17, or 33×33 kernels), which are then independently processed through subsequent feature clustering (see Section 4.3) and weight prediction stages(see Section 5), thereby establishing a tightly integrated edge-aware filtering pipeline.

4.3. Pixel region division

Within each kernel size group, we conduct secondary clustering based on depth-channel geometric features, including pixel depth and normal gradients. This partitions the pixels into multiple clusters, where we assume the filter weights within each cluster exhibit similar characteristics. The weight distribution for pixels in each cluster can then be efficiently retrieved using a compact LUT with only a few entries.

For a central pixel p, we construct a feature vector $\mathbf{F}_p = [M^{\text{Depth}}(p), \theta^{\text{Depth}}(p)]$, where $M^{\text{Depth}}(p)$ denotes the depth gradient magnitude (Eq. (2)) and $\theta^{\text{Depth}}(p) = \frac{\arctan 2(G_y^{Depth}, G_x^{Depth}) + \pi}{2\pi}$ represents the normalized gradient direction. These features capture local geometric coherence: M^{Depth} quantifies surface discontinuity strength, while θ^{Depth} encodes orientation consistency.

We employ K-means clustering (a classical unsupervised method) [18] to partition each kernel group into $\mathbf{F}_p$ based sub-clusters by minimizing intra-class variance. Each resulting cluster individually guides dynamic weight allocation in the subsequent prediction network. This two-stage clustering enables more precise weight assignment in the prediction network, concentrating computational resources on perceptually critical regions within each kernel-size group.

5. Weight prediction

In mobile real-time ray tracing denoising tasks, the dynamic prediction of filter kernel weights must simultaneously meet the requirements of high quality and low latency. Based on edge-aware clustering results, this section proposes a collaborative weight prediction framework that combines precomputed LUTs with a lightweight MLP network. Specifically, we train a dedicated lightweight MLP for each cluster using only the pixels within that specific cluster. Each network operates independently with no shared parameters. This fine-grained, cluster-specific MLP training effectively captures local weight distributions. Finally, the weights predicted by all these per-cluster MLPs are quantized into our unified 5D LUT. The framework constructs a nonlinear mapping from low-dimensional features to filter weights through offline precomputation of a global LUT. During denoising, weights are efficiently retrieved via quantized feature indexing, eliminating the computational overhead of neural network inference.

5.1. Weight prediction network

Inspired by Kalantari et al. [10], the proposed method employs a lightweight MLP for real-time denoising on mobile ray tracing hardware. The network structure is illustrated in Fig. 3 Unlike SVGF, which relies on computationally intensive variance and weight estimation, our network directly leverages G-buffer features to predict denoising parameters through learned correlations, eliminating complex variance and weight calculations to improve denoising efficiency.

Input Representation. The network accepts a 5-channel input tensor derived from the G-buffer. It contains

1. Luminance: A single channel converted from RGB via $L = 0.2126R + 0.7152G + 0.0722B$, capturing photometric intensity critical for noise perception.
2. Surface Normals: Three channels (XYZ components in tangent space) encoding local geometric orientation.
3. Depth: A single channel representing view-space Z-values, normalized to [0, 1] for numerical stability.

Network Architecture. The MLP employs a three-hidden-layer structure, balancing expressivity and latency. It contains

1. First Hidden Layer (128 neurons, ReLU): Extracts high-dimensional feature interactions via $\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$. Capture the nonlinear coupling among luminance variations (ΔL), surface normal discontinuities (ΔN), and depth variations (ΔZ).

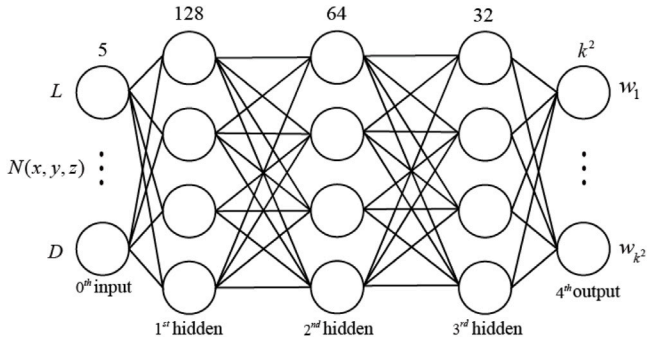


Fig. 3. The weight prediction network consists of an input layer, an output layer, and three hidden layers, with ReLU serving as the activation function for all layers. For clarity, the diagram illustrates partial node connections within each layer.

2. Second Hidden Layer (64 neurons, ReLU): Compresses features through $\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{x} + \mathbf{b}_2)$. Compresses features by 50%, filtering noise-sensitive components while retaining salient patterns through hierarchical abstraction.

3. Third Hidden Layer (32 neurons, ReLU): Projects features to latent weight space using $\mathbf{h}_3 = \text{ReLU}(\mathbf{W}_3\mathbf{x} + \mathbf{b}_3)$.

The output layer generates raw filter weights $\mathbf{w} \in \mathbb{R}^{k^2}$, where k^2 corresponds to the dynamically selected kernel size from Section 4.2.

Loss Function. The network is trained to approximate SVGF's reference weights using mean squared error (MSE):

$$\mathcal{L} = \frac{1}{k^2}\sum_{i=1}^{k^2}\left(w_i - w_{\text{SVGF},i}\right)^2 \tag{8}$$

where w_i denotes the normalized weights predicted by the network, and $w_{\text{SVGF},i}$ represents the reference weights generated by the conventional SVGF algorithm. $w_{\text{SVGF},i}$ are derived from depth, world-space normals, and input luminance, formulated as $w_{\text{SVGF},i} = w_z \cdot w_n \cdot w_l$. Notably, We apply the L2 loss directly to the filter weights, as our goal is to rapidly construct an explicit filter kernel. Compared to the L1 loss, the squared term in the L2 loss more strongly penalizes significant deviations in the weight distribution, thereby more effectively correcting severe errors in the filter kernel [19]. This is crucial for preserving the physical properties of the filter kernel, such as penalizing depth discontinuities and rewarding normal consistency.

5.2. LUT-based weight prediction

To eliminate real-time inference overhead entirely, we propose a hardware-friendly LUT architecture for instantaneous weight prediction on mobile GPUs. The core innovation lies in quantizing the continuous 5D feature space – surface normals $\mathbf{n} = (n_x, n_y, n_z)$, depth d, and luminance L – into discrete indices, enabling direct weight retrieval from a precomputed table.

Normal Quantization. Normals are converted to spherical coordinates (θ, ϕ), where $\theta = \arccos(n_z)$ and $\phi = \arctan_2(n_y, n_x)$, then uniformly quantized into N_n bins:

$$Q_\theta = \left\lfloor \frac{\theta}{2\pi/N_n} \right\rfloor, \quad Q_\phi = \left\lfloor \frac{\phi + \pi}{\pi/N_n} \right\rfloor, \tag{9}$$

where Q_θ and Q_ϕ denote the quantization indices for the polar angle (θ) and azimuthal angle (ϕ) respectively, while N_n represents the number of quantization bins for the normal components, ensuring isotropic sampling on the unit sphere.

Depth Quantization. Depth values $d \in [d_{\min}, d_{\max}]$ are log-scaled and quantized into N_d bins to prioritize near-field precision:

$$Q_d = \left\lfloor N_d \cdot \frac{\log(d) - \log(d_{\min})}{\log(d_{\max}) - \log(d_{\min})} \right\rfloor. \tag{10}$$

where $d_{\min}$ and $d_{\max}$ define the scene's depth range, Q_d denotes the depth quantization index, and N_d represents the number of depth quantization bins. This aligns with human depth perception, preserving accuracy at occlusion boundaries.

Luminance Quantization. Luminance L is linearly quantized after tone mapping to [0, 1]:

$$Q_L = \lfloor N_L \cdot L \rfloor \tag{11}$$

where Q_L denotes the luminance quantization index, and N_L represents the number of luminance quantization bins. The selection of N_L should aim to distinguish key luminance ranges (e.g., shadows, mid-tones) while aligning with the color gamut constraints of mobile displays.

The quantized indices $I = (Q_c, Q_\theta, Q_\phi, Q_L, Q_d)$ map to a precomputed 5D tensor $\mathbf{T}$ storing filter weights for all kernel sizes $k \in \{9, 17, 33\}$, where Q_c denotes grouping clustering result. Weights are generated offline by inferring the MLP (Section 5.1) across all quantized feature combinations:

$$\mathbf{W}_I^{(k)} = \text{MLP}(\mathbf{F}_I), \tag{12}$$

where $\mathbf{F}_I$ represents the centroid feature vector of bin I. The full-precision model maintains a precomputed tensor $\mathbf{T}$ serving as an offline reference during training, with a total memory footprint of 5.7 GB. This uncompressed, high-precision LUT functions as a universal benchmark for all rendered scenes, ensuring consistent performance evaluation across diverse rendering conditions. For mobile deployment, we implement an optimization strategy: weights are stored at FP16 precision, while retaining only frequently accessed bins (e.g., those corresponding to dominant materials and depth ranges). This optimization reduces the memory footprint of the universal LUT to approximately 2.5 GB. The compressed LUT retains its universality, is fully pre-stored on the device without requiring streaming, and operates as a self-contained module loaded only once. At runtime, weights are retrieved in $\mathcal{O}(1)$ time, eliminating real-time computation.

6. Implementation

Dataset. Our experiments are conducted on the NoiseBase dataset [20], an open-source benchmark specifically designed for Monte Carlo rendering denoising tasks. The dataset encompasses a diverse range of ray-traced rendering scenarios, including indoor/outdoor environments, complex materials (e.g., metals, glass, fur), and dynamic lighting conditions. Each scene provides 1spp noisy images, G-buffers and Ground truth reference images, enabling spatiotemporal denoiser training and validation. For evaluation, we select 5 scenes, each containing a 160-frame animation sequence at a resolution of 1080 × 1920, totaling 800 frames. For three scenes (Zero-day [21], Bistro1, Bistro3), we randomly split 80% for training and 20% for testing. The Bistro2 and Kitchen scene is entirely excluded from training and reserved for testing, ensuring rigorous evaluation of model generalization. Among the training scenes, Bistro1 feature diverse material compositions, and Bistro3 is a multi-light source hybrid light field. Meanwhile, Zero-day contains intricate geometric details and high-reflectance surfaces.

Training. The proposed method implements the backbone network and output convolutional layers in PyTorch, with filtering operations optimized in CUDA. Training employs the Adam optimizer with an initial learning rate of 0.001, decayed by 50% every 128 epochs. We use a batch size of 32 and train on full-resolution 1920 × 1080 patches, augmented with random flips and rotations. The experimental setup operates on an NVIDIA RTX 4060 GPU and Intel Core i9-14900HX CPU, with full model training requiring approximately 8 h. During training, the network converges after approximately 500 epochs.

Mobile Deployment. Experiments are conducted on a Qualcomm Snapdragon 768G mobile platform featuring an Adreno 620 GPU with hardware-accelerated ray tracing and FP16 support. Core algorithm modules are implemented via Unity shaders, with screen refresh rate fixed at 60 Hz to emulate typical mobile resource constraints.

7. Evaluation

In this section, we evaluate our novel denoising method across multiple metrics, including final image quality, computational performance, and power consumption, and compare it with SVGF and BMFR to highlight the advantages of our method. All methods are implemented under the spatial domain filtering framework, and the test scenarios are unseen scenes not encountered during the training phase. Our evaluation focuses on 1spp images, as our design decisions are specifically optimized for this sampling rate. Finally, we conduct ablation studies on the edge-aware module and LUT bin counts to demonstrate their critical importance to the entire system.

7.1. Image quality

We conducted a comparative analysis of five benchmark scenes (Bistro1-3, Zero-day Scene, and Kitchen Scene) against classic SVGF and BMFR algorithms. Fig. 4 presents selected frames from these animations for detailed comparison, with insets highlighting key differences in the results. The findings demonstrate consistent improvements in both quantitative metrics and visual quality with our method.

Quantitatively, our method achieves lower RMSE values in all test cases, even the BMFR algorithm which outperforms SVGF. Visually, the proposed method effectively eliminates Monte Carlo noise without introducing the global blurring characteristic of SVGF, as evidenced in Bistro1's high dynamic range (HDR) lighting conditions. The edge-aware filtering preserves fine geometric details (such as the bookshelf contours in Bistro2) while avoiding SVGF's halo artifacts, and successfully retains high-frequency textures (like the carpet pattern in Bistro3) that SVGF and BMFR tends to oversmooth.

Our method demonstrates unique advantages in complex lighting environments, as demonstrated by its superior handling of mixed illumination and multiple reflections in the Zero-day scene, as well as robust denoising across diverse surface materials in the Kitchen scene. These results validate that our adaptive kernel strategy achieves a better noise-detail tradeoff compared to SVGF's fixed-filter method.

As for the BMFR algorithm, its purely mathematical fitting method leads to particularly poor performance in complex scenes with fine details, as clearly observed in both Bistro2 and Bistro3 scenarios.

7.2. Performance

Table 1 summarizes the average frame rate, power consumption, and per-stage processing time over 1,000 frames in the Bistro scene on mobile platforms. As evident from Fig. 4, our algorithm demonstrates significant advantages over conventional SVGF and algebra-based BMFR methods in both real-time performance and energy efficiency—precisely aligning with the core optimization objectives of this study.

On the Qualcomm Snapdragon 768G platform, our proposed dynamic kernel allocation strategy and precomputed LUT mechanism deliver an average 67.42% higher frame rate and 16.80% lower power consumption compared to SVGF, while maintaining superior visual quality. Furthermore, when benchmarked against the more real-time-friendly BMFR algorithm, our solution still achieves an average 10.86% frame rate improvement and 5.45% power reduction.

This breakthrough validates the effectiveness of our co-optimized framework for mobile ray tracing scenarios, providing a practical solution for real-time rendering under strict power constraints.

Table 1
Per-stage execution time, performance, and power consumption are shown. Our algorithm consistently outperforms previous work.

Processing Stage	Ours
Edge extraction (ms)	2.08
Feature calculations (ms)	5.04
LUT (ms)	0.64
Convolution (ms)	10.58
Frame rate (fps)	55
Power (W)	1.88

Table 2
Ablation study on rendering quality and computational efficiency across mobile-optimized architectures.

Method	Ours	w/o E	8 Bins	4 Bins
SSIM	**0.792**	0.705	0.778	0.743
PSNR	**23.95**	22.55	23.43	22.76
RMSE	**0.063**	0.075	0.067	0.073
TIME (ms)	19.32	23.61	19.05	**18.63**

7.3. Ablation study

To analyze the contribution of each component, we conducted comprehensive ablation studies. First, we removed the edge-aware module (w/o), eliminating both edge density computation and adaptive kernel size determination, leaving only clustering to test a global uniform-kernel denoising filter. We then investigated the impact of feature quantization granularity by partitioning features into 4 and 8 bins respectively. Table 2 presents average metrics across multiple test scenarios, while Figs. 5 and 6 provide side-by-side qualitative comparisons.

The removal of the edge-aware module resulted in significant performance degradation across all metrics, with particularly notable deterioration in high-frequency detail preservation. As shown in the Edge-Aware-removed column of Fig. 5, road surfaces and vegetable edges appear blurred, and halo artifacts re-emerge due to the lack of edge guidance. Our experiments demonstrate that the edge-aware mechanism with dynamic kernel size allocation enables simultaneous noise suppression and detail preservation in complex scenes, establishing it as a critical pillar for high-quality denoising.

To examine how feature quantization affects denoising quality, we reduced the default 16 quantization bins to 8, which takes up about 1.5G of memory. The SSIM and PSNR metrics remained virtually unchanged, suggesting that moderate quantization has limited impact on overall performance. However, further reduction to 4 bins caused significant declines in both SSIM and RMSE, manifesting as erroneous weight assignments in high-complexity regions like book textures and sign details. The essence of quantization binning lies in dimensionality reduction for accelerated LUT indexing. Nevertheless, excessive coarsening (e.g., 4 bins) compromises local feature representation, leading to mismatches between filter weights and actual distributions. Our findings indicate that the default 16-bin quantization achieves the optimal balance between mobile hardware overhead and quality preservation, while 8 bins may serve as a viable alternative for severely resource-constrained scenarios.

7.4. Comparison with E-SVGF

Based on the core methodology proposed in this work, we extend our comparative analysis by introducing E-SVGF as a more advanced baseline. This novel method integrates edge-aware filter kernels with the original SVGF weights, significantly improving its capacity to



Fig. 4. Our denoiser is compared with the classic SVGF and BMFR algorithms in five benchmark scenes. Our method demonstrates superior performance across all scenes in terms of quantitative metrics. In the illustrative figures for each scene, red arrows highlight artifacts produced by SVGF and BMFR denoising, while our method achieves results closer to the ground truth.

preserve fine details.

To rigorously evaluate performance and efficiency, we conducted comparative experiments between our proposed data-driven method and E-SVGF in the challenging bistro1 scene (see Fig. 7). The results demonstrate that, compared to the E-SVGF algorithm, our data-driven method achieves an average frame rate improvement of 29.44% and a power consumption reduction of 8.92%, while maintaining comparable visual quality. Despite E-SVGF's exceptional denoising performance, its high computational intensity renders it impractical for mobile devices. In contrast, our data-driven method achieves comparable denoising quality without such high computational overhead, rendering it more suitable for mobile deployment.

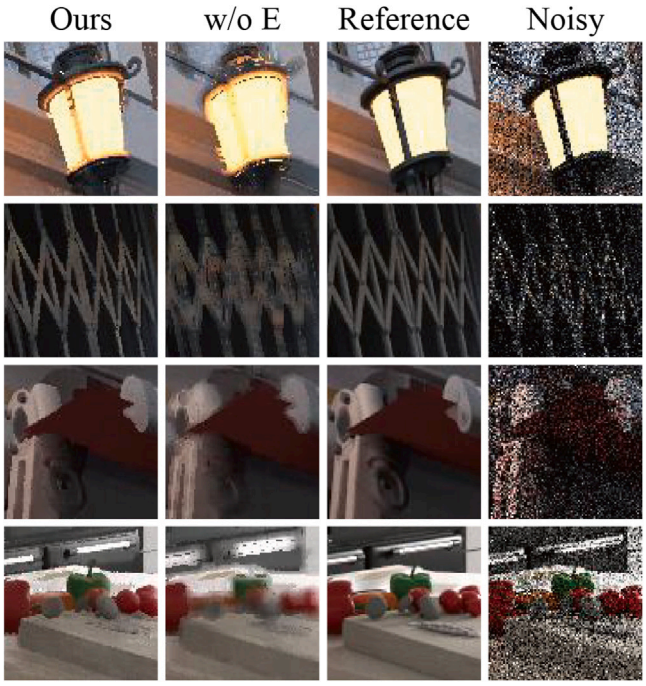


Fig. 5. Ablation on edge-Aware module. This study compares the denoising performance of the full model against a version with the edge-aware module removed ("w/o E"). The last two columns display the reference image and noisy input.

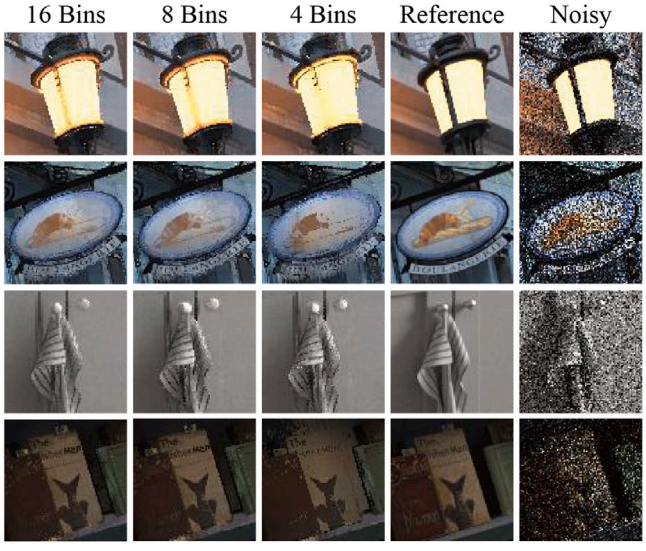


Fig. 6. Ablation on LUT quantization bins. This study evaluates the impact of varying numbers of LUT quantization bins on image denoising accuracy. The complete filter uses 16 bins. The last two columns display the reference image and noisy input.

8. Limitations

The edge-aware denoising architecture proposed in this study demonstrates significant advantages in mobile ray-traced rendering, particularly in noise suppression efficiency and edge detail preservation compared to traditional methods. However, it exhibits theoretical and practical limitations.

Sensitivity to Input Data Quality. Edge detection relies heavily on noise-free G-buffers, including normals, depth, and albedo channels. However, inherent rendering artifacts – such as aliasing in low-precision depth buffers or interpolation errors in normal maps – can amplify false edge detection. For instance, staircase-like noise in depth buffers may be misclassified as geometric edges, leading to incorrect filter weight allocation. Furthermore, abrupt illumination changes (e.g., flickering light sources) may introduce spurious gradients in albedo channels, further distorting material edge extraction. As shown in Fig. 8(a), sudden light exposure on the pillar caused edge extraction failure, resulting in noticeable denoising blur.

Challenges in Extreme Dynamic Scenes. The reuse of edge density relies on accurate motion vectors. In scenarios with rapid object rotation, deformation (e.g., fluttering flags), or complex non-rigid motion, motion compensation errors accumulate, causing misalignment between reprojected historical edges and current-frame edges. This manifests as ghosting artifacts or fragmented edges. In Fig. 8(b), two uniformly colored emissive spheres undergoing complex interactive motion demonstrate this issue: motion error accumulation introduces white artifacts within the pink sphere, along with a distinct pink-white boundary, significantly degrading denoising quality. Furthermore, in regions with extremely dense edge structures (e.g., foliage or chainmail), the system defaults to using the minimum kernel size (9×9). While computationally efficient, this may provide insufficient spatial support to suppress high-frequency noise, resulting in residual artifacts.

Generalization Bottlenecks in Quantization and LUT. Although LUT-based weight prediction significantly reduces computational overhead, its performance is limited by the coverage of offline training data. For materials unseen in the training set, the LUT may generate suboptimal weights, leading to localized over-smoothing or residual noise after denoising. As shown in Fig. 8(c), this limitation becomes apparent when denoising glass materials: the LUT's limited exposure to such specular surfaces during training results in noticeable quality degradation, typically manifesting as excessive blurring. Additionally, while the quantization strategy in this study optimizes storage efficiency through techniques like uniform binning, the inherent trade-off between feature dimensions and quantization precision still restricts its scalability in ultra-high-definition rendering (e.g., 4 K resolution). Specifically, finer-grained feature partitioning at higher resolutions may cause LUT storage requirements to grow exponentially.

9. Conclusion

To address the challenge of denoising low-sample-rate path-traced images on power-constrained mobile devices, we propose a lightweight and efficient architecture that combines edge-adaptive filtering, data-driven weight prediction, and motion-aware reuse. Our method adapts the filter kernel based on local edge information, predicts the weights of a compact MLP via a quantized LUT, and reuses edge density estimated from motion vectors to reduce the cost of real-time edge detection. On the Qualcomm Snapdragon 768G platform, our implementation achieves 55 FPS – representing a 67.42% increase in frame rate and a 16.80% reduction in power consumption compared to a conventional SVGF baseline – while achieving higher visual fidelity. These results validate the method's suitability for interactive path tracing on mobile hardware. Although performance remains robust under typical conditions, challenges persist in scenes with extreme material properties or very high resolutions. Future work will investigate adaptive motion compensation and online LUT refinement to improve generalization and scene adaptability. To conclude, this work demonstrates a practical denoising solution that enables real-time path tracing on mobile platforms, facilitating the transition from rasterization to physically-based rendering in next-generation mobile graphics.

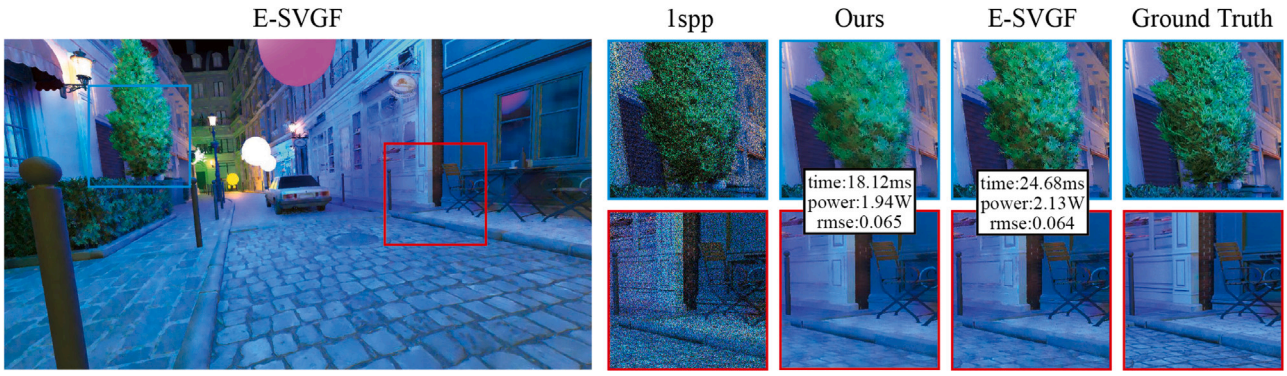


Fig. 7. Performance of E-SVGF on mobile platforms. Our method achieves comparable results to E-SVGF while significantly reducing computational overhead.

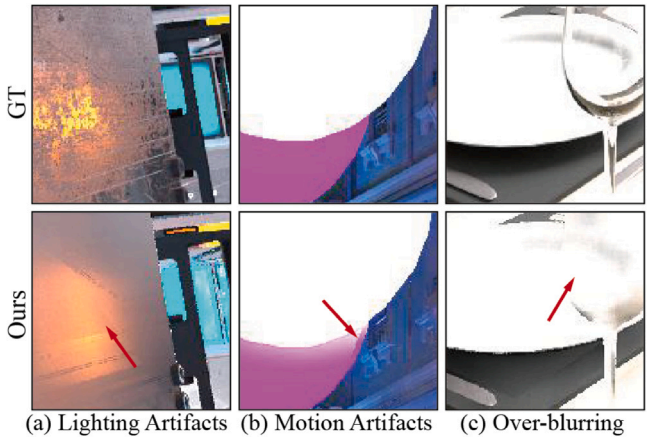


Fig. 8. Limitations of our denoiser. (a) shows abrupt illumination changes introduce spurious gradients, causing noticeable denoising blur. (b) shows motion artifacts emerge during rapid interaction between two spheres. (c) shows over-blurring occurs on rare glass materials due to limited training exposure.

CRediT authorship contribution statement

Haosen Fu: Writing – original draft, Visualization, Validation, Project administration, Methodology, Formal analysis, Conceptualization. **Mingcong Ma:** Resources, Data curation. **Junqiu Zhu:** Writing – review & editing. **Lu Wang:** Writing – review & editing. **Yanning Xu:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partially supported by the National Natural Science Foundation of China (No. 62272275), the Taishan Scholars Program (No. tsqn202312231), Qilu University of Technology (Shandong Academy of Sciences) Faculty of Computer Science and Technology Pairing Program (No. 2024JDJH13).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.gmod.2025.101301.

Data availability

In the paper, I have given references to the open source dataset.

References

[1] C. Schied, A. Kaplanyan, C. Wyman, A. Patney, C.R.A. Chaitanya, J. Burgess, et al., Spatiotemporal variance-guided filtering: Real-time reconstruction for path-traced global illumination, Proc. High Perform. Graph. (2017) URL: https://api.semanticscholar.org/CorpusID:202716856.

[2] M. Zwicker, W. Jarosz, J. Lehtinen, B. Moon, R. Ramamoorthi, F. Rousselle, et al., Recent advances in adaptive sampling and reconstruction for Monte Carlo rendering, Comput. Graph. Forum (Proceedings Eurographics - State the Art Reports) 34 (2) (2015) 667–681, 10/f7k6kj.

[3] C.H. Lampert, O. Wirjadi, An optimal nonorthogonal separation of the anisotropic Gaussian convolution filter, IEEE Trans. Image Process. 15 (2006) 3501–3513, URL: https://api.semanticscholar.org/CorpusID:14816789.

[4] E. Eisemann, F. Durand, Flash photography enhancement via intrinsic relighting, ACM Trans. Graph. 23 (3) (2004) 673–678, http://dx.doi.org/10.1145/1015706.1015778.

[5] H. Dammertz, D. Sewtz, J. Hanika, H.P.A. Lensch, Edge-avoiding À-Trous wavelet transform for fast global illumination filtering, in: Proceedings of the Conference on High Performance Graphics, HPG '10, Eurographics Association, Goslar, DEU, 2010, pp. 67–75.

[6] K. He, J. Sun, X. Tang, Guided image filtering, IEEE Trans. Pattern Anal. Mach. Intell. 35 (6) (2013) 1397–1409, http://dx.doi.org/10.1109/TPAMI.2012.213.

[7] M. Mara, M. McGuire, B. Bitterli, W. Jarosz, An efficient denoising algorithm for global illumination, in: Proceedings of High Performance Graphics, HPG '17, Association for Computing Machinery, New York, NY, USA, 2017, http://dx.doi.org/10.1145/3105762.3105774.

[8] M. Koskela, K. Immonen, M. Mäkitalo, A. Foi, T. Viitanen, P. Jääskeläinen, H. Kultala, J. Takala, Blockwise multi-order feature regression for real-time path-tracing reconstruction, ACM Trans. Graph. 38 (5) (2019) http://dx.doi.org/10.1145/3269978.

[9] C. Schied, C. Peters, C. Dachsbacher, Gradient estimation for real-time adaptive temporal filtering, Proc. ACM Comput. Graph. Interact. Tech. 1 (2) (2018) http://dx.doi.org/10.1145/3233301.

[10] N.K. Kalantari, S. Bako, P. Sen, A machine learning approach for filtering Monte Carlo noise, ACM Trans. Graph. 34 (4) (2015) http://dx.doi.org/10.1145/2766977.

[11] C.R.A. Chaitanya, A.S. Kaplanyan, C. Schied, M. Salvi, A. Lefohn, D. Nowrouzezahrai, et al., Interactive reconstruction of Monte Carlo image sequences using a recurrent denoising autoencoder, ACM Trans. Graph. 36 (4) (2017) http://dx.doi.org/10.1145/3072959.3073601.

[12] T. Vogels, F. Rousselle, B. Mcwilliams, G. Röthlin, A. Harvill, D. Adler, et al., Denoising with kernel prediction and asymmetric loss functions, ACM Trans. Graph. 37 (4) (2018) http://dx.doi.org/10.1145/3197517.3201388.

[13] B. Xu, J. Zhang, R. Wang, K. Xu, Y.-L. Yang, C. Li, et al., Adversarial Monte Carlo denoising with conditioned auxiliary feature modulation, ACM Trans. Graph. 38 (6) (2019) http://dx.doi.org/10.1145/3355089.3356547.

[14] J. Hasselgren, J. Munkberg, M. Salvi, A. Patney, A. Lefohn, Neural temporal adaptive sampling and denoising, Comput. Graph. Forum 39 (2) (2020) 147–155, http://dx.doi.org/10.1111/cgf.13919, URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13919.

[15] arm, Arm® GPU best practices developer guide, 2023, URL: https://developer.arm.com/documentation/101897/0304/.

[16] Oppo Newsroom, One of the industry's first open ray tracing solution: OPPO brings about realistic rendering to more applications, 2022, URL: https://www.oppo.com/en/newsroom/stories/first-open-ray-tracing-oppo-qualcomm-snapdragon/.

[17] I. Sobel, G. Feldman, A 3×3 isotropic gradient operator for image processing, Pattern Classif. Scene Anal. (1973) 271–272.

[18] A.M. Ikotun, A.E. Ezugwu, L. Abualigah, B. Abuhaija, J. Heming, K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data, Inform. Sci. 622 (2023) 178–210, http://dx.doi.org/10.1016/j.ins.2022.11.139, URL: https://www.sciencedirect.com/science/article/pii/S0020025522014633.

[19] H. Zhao, O. Gallo, I. Frosio, J. Kautz, Loss functions for image restoration with neural networks, IEEE Trans. Comput. Imaging 3 (1) (2017) 47–57, http://dx.doi.org/10.1109/TCI.2016.2644865.

[20] M. Balint, K. Wolski, K. Myszkowski, H.-P. Seidel, R. Mantiuk, Neural partitioning pyramids for denoising Monte Carlo renderings, in: ACM SIGGRAPH 2023 Conference Proceedings, SIGGRAPH '23, Association for Computing Machinery, New York, NY, USA, 2023, http://dx.doi.org/10.1145/3588432.3591562.

[21] Mike Winkelmann, Zero-day, open research content archive (ORCA), 2019, URL: https://developer.nvidia.com/orca/beeple-zero-day.